

CHƯƠNG 3. TÍNH KẾ THỪA

TS. Phạm Minh Hoàn

Viện Công nghệ thông tin Kinh tế – Đại học Kinh tế Quốc dân

Email: hoanpm@neu.edu.vn

MỤC TIÊU

- *Trình bày khái niệm tính kế thừa, lớp cơ sở, lớp dẫn xuất và phạm vi của kế thừa.*
- *Trình bày sự kế thừa các thành phần và phạm vi truy nhập đến các thành phần của lớp cơ sở.*
- *Trình bày tính kế thừa của các hàm đặc biệt của lớp: hàm tạo và hàm huỷ.*

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

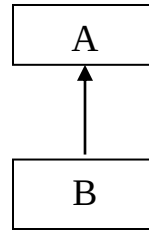
KHÁI NIỆM

- Kế thừa là một trong các khái niệm cơ sở của phương pháp lập trình hướng đối tượng.
- **Tính kế thừa cho phép định nghĩa các lớp mới từ các lớp đã có.**
- Lớp đã có được gọi là **lớp cơ sở**, lớp mới được gọi là **lớp dẫn xuất**.
- Một lớp có thể là lớp cơ sở cho nhiều lớp dẫn xuất khác nhau.
- Lớp dẫn xuất sẽ kế thừa một số thành phần (dữ liệu và hàm) của lớp cơ sở, đồng thời có thêm những thành phần mới.

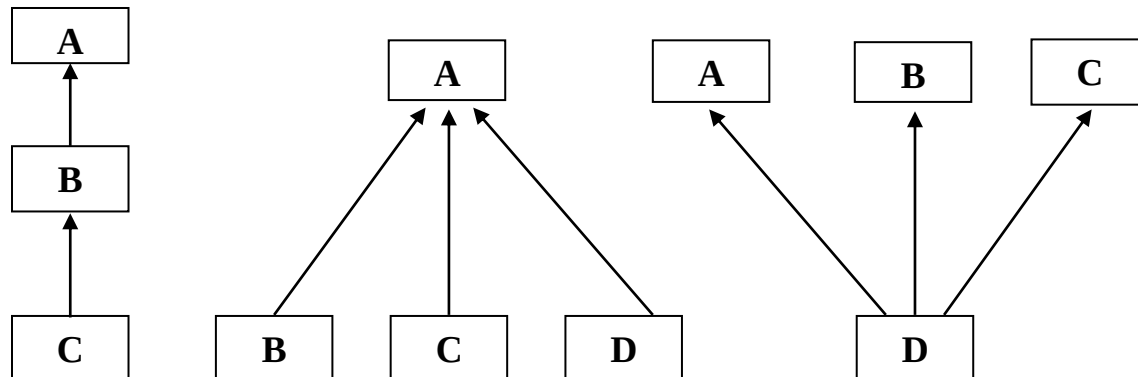
SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

PHÂN LOẠI

○ Đơn kế thừa



○ Đa kế thừa



SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

ĐỊNH NGHĨA KẾ THỪA

- Giả sử đã định nghĩa các lớp A, B, Cú pháp để xây dựng lớp Z dẫn xuất từ các lớp A, lớp B, ... như sau:

```
class Z: private/public A [, private/public B[...]]  
{  
    private:  
        // Khai báo các thuộc tính của lớp Z  
    public:  
        // Định nghĩa các hàm thành phần của lớp Z  
};
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

KẾ THỪA PRIVATE VÀ PUBLIC

- Kế thừa theo kiểu **public** thì tất cả các thành phần **public** của lớp cơ sở cũng là thành phần **public** của lớp dẫn xuất.
- Kế thừa theo kiểu **private** thì tất cả các thành phần **public** của lớp cơ sở sẽ trở thành các thành phần **private** của lớp dẫn xuất.

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

KẾ THỪA PRIVATE VÀ PUBLIC

- Trong cả hai trường hợp ở trên thì thành phần **private** của lớp cơ sở là không được kế thừa. Lớp dẫn xuất không cho phép truy nhập đến các thành phần **private** của lớp cơ sở.
- Lớp dẫn xuất không cho phép kế thừa các **hàm tạo**, **hàm hủy** và **toán tử gán** của lớp cơ sở.

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

TRUY XUẤT ĐẾN THÀNH PHẦN

- Thành phần của lớp dẫn xuất bao gồm:
 - Các thành phần khai báo trong lớp dẫn xuất.
 - Các thành phần mà lớp dẫn xuất thừa kế từ các lớp cơ sở.
- Quy tắc sử dụng các thành phần trong lớp dẫn xuất:

Tên_đối_tượng.Tên_lớp::Tên_thành_phần

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ

```
class A
{
    public: int n;
    void nhap()
    {
        cout<<"\n Nhap n = ";
        cin>>n;
    }
};
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ

```
class B: public A
{
    public: int m;
    void nhap()
    {
        cout<<"\n Nhap m = ";
        cin>>m;
    }
};
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ

- Xét khai báo: B ob;
- Khi đó:
 - ob.m là thuộc tính m khai báo trong B
 - ob.n là thuộc tính n thừa kế từ lớp A
 - ob.B::nhap() là hàm nhap() định nghĩa trong lớp B
 - ob.A::nhap() là hàm nhap() định nghĩa trong lớp A
 - ob. nhap().

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

CHÚ Ý

- Nếu truy xuất đến thành phần của lớp dẫn xuất mà không dùng tên lớp chương trình dịch phải tự phán đoán để biết thành phần đó thuộc lớp nào.
- Thứ tự truy xuất: lớp dẫn xuất, các lớp có quan hệ gần với lớp dẫn xuất sẽ được xét trước, các lớp quan hệ xa hơn xét sau.
- Trong trường hợp thành phần đang xét có mặt đồng thời trong 2 lớp cơ sở có cùng một cấp quan hệ với lớp dẫn xuất, chương trình dịch sẽ đưa ra thông báo lỗi.

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 1

```
#include <iostream>
#include <conio.h>
#include <stdlib.h>
class A
{
    int a;
protected:
    int b;
public:
    void set_ab();
    int get_a(void);
    void show_a(void);
    void set_b(){b=20;}
};
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 1

```
class B: public A
{
    int c;
    public:
    void mul(void);
    void display(void);
};
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 1

```
void A::set_ab(void)
{
    a = 5;
    b = 10;
}
int A::get_a()
{
    return a;
}
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 1

```
void A::show_a()
{
    cout << "a = " << a << " \n";
}
void B::mul()
{
    c = b*get_a();
}
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 1

```
void B::display()
{
    cout << "a = " << get_a() << "\n";
    cout << "b = " << b << "\n";
    cout << "c = " << c << "\n";
}
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 1

```
main()
{
    B d;
    d.set_ab(); //Ke thua tu A
    d.mul();
    d.show_a(); //Ke thua tu A
    d.display();
    //d.b = 20;
    d.set_b();
    d.mul();
    d.display();
    getch();
}
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 1

○ Chương trình cho kết quả:

$a = 5$

$a = 5$

$b = 10$

$c = 50$

$a = 5$

$b = 20$

$c = 100$

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 2

```
#include <conio.h>
#include <iostream.h>
class Diem
{
    private:
        double x, y;
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 2

```
public:
```

```
void nhap()
```

```
{
```

```
    cout<<"\n x = "; cin>>x;
```

```
    cout<<"\n y = "; cin>>y;
```

```
}
```

```
void hienthi()
```

```
{
```

```
    cout<<"\n x = "<<x<<" y = "<<y;
```

```
}
```

```
};
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 2

```
class Hinhtron: public Diem
{
    private: double r;
    public:
        void nhap_r()
        { cout<<"\n r = "; cin>>r; }
        double get_r()
        { return r; }
};
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 2

```
main()
{
    Hinhtron h;
    clrscr();
    cout<<"\n Nhap toa do tam va ban kinh hinh tron";
    h.nhap(); h.nhap_r();
    cout<<"\n Hinh tron co tam:";h.hienthi();
    cout<<"\n Co ban kinh = " << h.get_r();
    getch();
}
```

SỰ DẪN XUẤT VÀ TÍNH KẾ THỪA

VÍ DỤ 2

○ Chương trình cho kết quả:

Nhap toa do tam va ban kinh hình tron

$$x = 2$$

$$y = 3$$

$$r = 10$$

Hình tron co tam:

$$x = 2 \ y = 3$$

$$\text{Co ban kinh} = 10$$

KHAI BÁO PROTECTED

ĐẶT VẤN ĐỀ

- Thành phần khai báo private trong lớp cơ sở không được kế thừa trong lớp dẫn xuất. Có thể giải quyết vấn đề này bằng cách:
 - Sử dụng các phương thức của lớp cơ sở để truy nhập đến các thuộc tính của chính lớp cơ sở đó.
 - Chuyển chúng sang vùng public.

KHAI BÁO PROTECTED

KHÁI NIỆM

- Các thành phần **protected** có phạm vi truy nhập rộng hơn so với các thành phần **private**.
- Các thành phần **protected** có phạm vi truy nhập hẹp hơn so với các thành phần **public**.

KHAI BÁO PROTECTED

KHÁI NIỆM

○ Các thành phần **protected**:

- Nếu kế thừa theo kiểu public thì các thành phần protected của lớp cơ sở sẽ trở thành các thành phần protected của lớp dẫn xuất.
- Nếu kế thừa theo kiểu private thì các thành phần protected của lớp cơ sở sẽ trở thành các thành phần private của lớp dẫn xuất.

KHAI BÁO PROTECTED

VÍ DỤ 1

```
#include <iostream.h>
#include <string.h>
#include <conio.h>
class M
{
    protected :
        int m;
    public :
        void getm(int x) {m=x;}
};
```

KHAI BÁO PROTECTED

VÍ DỤ 1

```
class N
{
    protected :
        int n;
    public :
        void getn(int y) {n=y;}
};
```

KHAI BÁO PROTECTED

VÍ DỤ 1

```
class P : public M,public N
{
    public :
        void display(void)
        {
            cout<<"m= "<<m<<endl;
            cout<<"n= "<<n<<endl;
            cout<<"m * n = "<<m*n<<endl;
        }
};
```

KHAI BÁO PROTECTED

VÍ DỤ 1

```
main()
{
    P ob;
    ob.getm(10);
    ob.getn(20);
    ob.display();
    getch();
}
```

KHAI BÁO PROTECTED

VÍ DỤ 1

- Chương trình cho kết quả như sau:

$$m = 10$$

$$n = 20$$

$$m*n = 200$$

KHAI BÁO PROTECTED

VÍ DỤ 2

- Viết chương trình quản lý kết quả thi của một lớp không quá 100 sinh viên. Chương trình gồm 3 lớp: lớp cơ sở sinh viên (sinhvien) chỉ lưu họ tên và số báo danh, lớp điểm thi (diemthi) kế thừa lớp sinh viên và lưu kết quả môn thi 1 và môn thi 2. Lớp kết quả (ketqua) lưu tổng số điểm đạt được của sinh viên.

KHAI BÁO PROTECTED

VÍ DỤ 2

```
#include <iostream>
#include <cstdlib>
#include <conio.h>
using namespace std;
class sinhvien
{
    char hoten[25];
protected:
    int sbd;
```

KHAI BÁO PROTECTED

VÍ DỤ 2

```
public:
    void nhap()
    { cout<<endl<<"Ho ten :";gets(hoten);
      cout<<endl<<"So bao danh :";cin>>sbd;
      cin.ignore(1);
    }
    void hienthi()
    { cout<<endl<<"So bao danh: "<<sbd;
      cout<<endl<<"Ho va ten sinh vien: "<<hoten;
    }
};
```

KHAI BÁO PROTECTED

VÍ DỤ 2

```
class diemthi : public sinhvien
{
    protected :
        float d1,d2;
    public :
        void nhap_diem()
        {
            cout<<"Nhap diem hai mon thi : \n";
            cin>>d1>>d2;
        }
}
```

KHAI BÁO PROTECTED

VÍ DỤ 2

```
void hienthi_diem()  
{  
    cout<<"Diem mon 1 :"<<d1<<endl;  
    cout<<"Diem mon 2 :"<<d2<<endl;  
}  
};
```

KHAI BÁO PROTECTED

VÍ DỤ 2

```
class ketqua : public diemthi
{
    float tong;
public :
    void display()
    {
        tong = d1+d2;
        hienthi();
        hienthi_diem();
        cout<<"Tong so diem : "<<tong<<endl;
    }
};
```

KHAI BÁO PROTECTED

VÍ DỤ 2

```
void main()
{
    int i,n; ketqua sv[100];
    cout<<"\n Nhap so sinh vien : ";
    cin>>n; cin.ignore(1);
```

KHAI BÁO PROTECTED

VÍ DỤ 2

```
for(i=0;i<n;++i)
{
    sv[i].nhap();
    sv[i].nhap_diem();
}
for(i=0;i<n;++i)
    sv[i].display();
getch();
}
```

KHAI BÁO PROTECTED

VÍ DỤ 3

- Viết chương trình mở rộng chương trình ở trên, trong đó ngoài kết quả thi hai môn, mỗi sinh viên còn có thể có điểm thưởng. Chương trình mở rộng thêm một lớp ưu tiên (uutien).

HÀM TẠO ĐỐI VỚI TÍNH KẾ THỪA

KHÁI NIỆM

- Các hàm tạo của lớp cơ sở là không được kế thừa.
- Một đối tượng của lớp dẫn xuất về thực chất có thể xem là một đối tượng của lớp cơ sở.
- Thứ tự thực hiện của các hàm tạo sẽ là: hàm tạo cho lớp cơ sở, rồi đến hàm tạo cho lớp dẫn xuất.

HÀM TẠO ĐỐI VỚI TÍNH KÊ THỪA

CÚ PHÁP

- Cú pháp định nghĩa hàm tạo để truyền tham số từ lớp dẫn xuất đến hàm tạo lớp cơ sở như sau:

Tên_lớp_dẫn_xuất(danh sách đối):

Tên_lớp_cơ_sở (danh sách đối)

{

 //thân hàm tạo của lớp dẫn xuất

};

HÀM TẠO ĐỐI VỚI TÍNH KẾ THỪA

VÍ DỤ

```
#include <iostream>
#include <cstdlib>
#include <conio.h>
using namespace std;
class Diem
{
    private: double x, y;
    public:
        Diem();
        Diem(double x1, double y1);
        void in();
};
```

HÀM TẠO ĐỐI VỚI TÍNH KÊ THỪA

VÍ DỤ

```
Diem::Diem()
{
    x=y=0.0;
}
Diem::Diem(double x1, double y1)
{
    x=x1; y=y1;
}
void Diem::in()
{
    cout<<"\nx="<<x<<"y="<<y;
}
```

HÀM TẠO ĐỐI VỚI TÍNH KÊ THỪA

VÍ DỤ

```
class Hinhtron: public Diem
{
    private: double r;
    public:
        Hinhtron()
        { r = 0.0; }
        Hinhtron(double x1,double y1,double r1): Diem(x1,y1)
        { r=r1; }
        double get_r()
        { return r; }
};
```

HÀM TẠO ĐỐI VỚI TÍNH KÊ THỪA

VÍ DỤ

```
main()
{
    Hinhtron h(2.5, 3.5, 8);
    cout<<"\n Hinh tron co tam:"; h.in();
    cout<<"\n Co ban kinh =" << h.get_r();
    getch();
}
```

HÀM HỦY ĐỐI VỚI TÍNH KẾ THỪA

KHÁI NIỆM

- Hàm hủy của lớp cơ sở cũng không được kế thừa.
- Hàm hủy được thi hành theo thứ tự ngược lại với hàm tạo. Nghĩa là, hàm hủy của lớp dẫn xuất thi hành trước hàm hủy của lớp cơ sở.

HÀM HỦY ĐỐI VỚI TÍNH KẾ THỪA

VÍ DỤ

```
#include <iostream.h>
#include <conio.h>
class CS
{
    public:
        CS()
        {cout<<"\nHam tao lop co so";}
        ~CS()
        {cout<<"\nHam huy lop co so";}
};
```

HÀM HỦY ĐỐI VỚI TÍNH KẾ THỪA

VÍ DỤ

```
class DX:public CS
{
    public:
        DX()
        {cout<<"\nHam tao lop dan xuat";}
        ~DX()
        {cout<<"\nHam huy lop dan xuat";}
};
```

HÀM HỦY ĐỐI VỚI TÍNH KẾ THỪA

VÍ DỤ

```
main()
{
    DX *ob = new DX;
    getch();
    delete ob;
    getch();
}
```

BÀI TẬP CHƯƠNG 3

- **Bài tập 1:** Xây dựng lớp Stack cho ngăn xếp kiểu int. Các hàm thành phần bao gồm: Hàm tạo mặc định, hàm hủy, hàm isEmpty() kiểm tra stack có rỗng không, hàm isFull() kiểm tra stack có đầy không, hàm push(), pop(), hàm in nội dung ngăn xếp. Sử dụng một mảng để thực hiện.
- **Bài tập 2:** Xây dựng lớp có tên là Stack với các thao tác cần thiết. Từ đó hãy dẫn xuất từ lớp stack để đổi một số nguyên dương sang hệ đếm bất kỳ.

TÓM TẮT CHƯƠNG 3

- *Khái niệm kế thừa, lớp cơ sở và lớp dẫn xuất, cách xây dựng lớp dẫn xuất, kế thừa private và public, kế thừa các thành phần dữ liệu, kế thừa phương thức, Phạm vi truy nhập đến các thành phần của lớp cơ sở.*
- *Khái niệm hàm tạo, hàm hủy đối với tính kế thừa, hàm tạo của lớp dẫn xuất, hàm hủy của lớp dẫn xuất.*